

03_ARCHITECTURE.md

TASK 03

SYSTEM ARCHITECTURE DESIGN

Status

Mandatory

Owner

Solution Architect

Supporting Guides

Project Director

Security Architect

Database Architect

Backend Lead

Frontend Lead

Principal Engineer

Estimated Time

1 to 2 Days

Dependencies

00_PROJECT_CONTEXT.md

00_ENGINEERING_RULES.md

00_PROJECT_MEMORY.md

03_SOLUTION_ARCHITECT.md

Mission

Transform the approved Software Requirements Specification into a complete, production-ready system architecture.

Design before development.

Every architectural decision should reduce complexity, improve security and support future scalability.

Do not begin backend or frontend implementation until architecture has been approved.

Objectives

Create a complete architecture for the application.

Define every layer.

Define every module.

Define every dependency.

Define every communication path.

Define every trust boundary.

Define every security boundary.

Produce diagrams and documentation that another engineer could immediately implement.

Architecture Principles

Follow:

Clean Architecture

Feature-Based Architecture

SOLID Principles

DRY

KISS

YAGNI

Separation of Concerns

Least Privilege

Secure by Design

Zero Trust

Every architectural decision should improve maintainability.

Technology Decisions

Confirm technology choices.

Frontend

Next.js 15

TypeScript

Tailwind CSS

shadcn/ui

Backend

Next.js Route Handlers

Server Actions

Prisma ORM

Database

Supabase PostgreSQL

Authentication

Supabase Authentication

Validation

Zod

Deployment

Docker

Vercel

No unapproved technologies should be introduced.

High-Level Architecture

Design the complete system.

Browser

↓

Next.js Application

↓

Middleware

↓

Authentication

↓

Authorization

↓

API Route Handlers



Service Layer



Repository Layer



Prisma ORM



Supabase PostgreSQL

Supporting Services

Storage

Logging

Audit

Notifications

Reporting

This architecture should remain stable throughout development.

Feature Modules

Design independent modules.

Authentication

Users

Roles

Artists

Parents

Coaches

Attendance

Performances

Injuries

Medical Records

Activities

Reports

Notifications

Dashboard

Settings

Audit Logs

Each module should own its own:

Pages

Components

Services

Validation

Database Access

Tests

Documentation

Avoid shared business logic.

Folder Structure

Produce a complete project structure.

Include:

app/

features/

components/

services/

repositories/

lib/

hooks/

types/

utils/

middleware/

prisma/

tests/

docs/

scripts/

config/

Explain the purpose of every folder.

Layer Responsibilities

Define responsibilities for:

Presentation Layer

Application Layer

Business Logic Layer

Data Access Layer

Infrastructure Layer

Utilities

No responsibility should overlap.

Request Lifecycle

Document every request.

Client

↓

Middleware

↓

Authentication

↓

Authorization

↓

Validation

↓

Business Logic

↓

Repository

↓

Database

↓

Response

Explain the purpose of every stage.

Authentication Architecture

Design authentication flow.

Login

Logout

Session Refresh

Protected Routes

Role Resolution

Session Expiration

Password Reset

Email Verification

Include sequence diagrams.

Authorization Architecture

Design RBAC.

Administrator

Coach

Parent

Artist

Document:

Permissions

Ownership Rules

Protected Resources

Privilege Boundaries

Never rely on frontend authorization.

Data Flow

Design complete data flow.

User Interaction

↓

Frontend

↓

API

↓

Business Logic

↓

Database

↓

Response

Produce:

Data Flow Diagram

Trust Boundary Diagram

Context Diagram

Container Diagram

Component Design

Design reusable components.

Cards

Forms

Tables

Charts

Dialogs

Navigation

Filters

Search

Pagination

Badges

Avatars

Notifications

Document how components interact.

State Management

Define:

Server Components

Client Components

Server Actions

Caching Strategy

Revalidation

Session State

Form State

Avoid unnecessary global state.

Performance Strategy

Design for:

Fast Rendering

Optimized Queries

Pagination

Lazy Loading

Caching

Minimal Bundle Size

Scalable Architecture

Performance should be considered before implementation.

Security Architecture

Integrate:

Authentication

Authorization

Validation

Rate Limiting

Secure Headers

Audit Logging

Environment Variables

Secrets Management

Error Handling

OWASP Controls

Document every security layer.

Scalability Strategy

Prepare for:

Multiple Academies

Additional Roles

Future Mobile App

Analytics

Scheduling

Payments

Notification Services

The architecture should support future expansion.

Diagrams

Produce:

System Architecture Diagram

Component Diagram

Deployment Diagram

Context Diagram

Container Diagram

Data Flow Diagram

Trust Boundary Diagram

Authentication Sequence Diagram

Authorization Sequence Diagram

Every diagram should include explanation.

Deliverables

Architecture Document

Architecture Decision Records

Folder Structure

Module Structure

Technology Decisions

Architecture Diagrams

Data Flow Design

Trust Boundaries

Request Lifecycle

Scalability Strategy

Performance Strategy

Security Architecture

Integration Strategy

Architecture Review

Verify:

Architecture is modular.

Dependencies are clean.

Business logic is isolated.

Security is layered.

Scalability is supported.

Folder structure is logical.

No circular dependencies exist.

Every module has one responsibility.

Diagrams are complete.

Documentation matches the design.

Update Project Memory

Mark:

Architecture Design

Completed

Record:

Architecture Decisions

Module Structure

Technology Decisions

Folder Structure

Known Risks

Future Considerations

Next Task

Database Design

Definition of Done

The architecture is approved.

Every module is defined.

Every dependency is documented.

Every layer has a clear responsibility.

Security is integrated into the architecture.

All diagrams are complete.

The architecture supports maintainability, scalability and secure development.

Backend and frontend implementation should begin only after the architecture has been reviewed and approved.